SOFTWARE ENGINEER · COMMERCIAL AND INDEPENDENT WORK

Commercial systems, native apps, and developer tools.

For more than six years, I have built and maintained software used in day-to-day business operations. Most of that code is private, so the cases below omit client names. The independent projects show what I build when I own the whole product.

● 6+ YEARS OF COMMERCIAL SOFTWARE DEVELOPMENT · PUBLIC CODE BELOW

Selected commercial work

Client and employer details are omitted because these systems are private. Technical descriptions are anonymised.

LOGISTICS INTEGRATION

Distribution routing and delivery exchange

2023–2025 · Senior 1C Developer · Logistics integration

BUSINESS PROBLEM

The route-planning service needed current orders and reference data from 1C. Planners also needed completed routes back in 1C.

MY WORK

I built data exchanges for products, vehicles, delivery points, warehouses, shipment lines, and route results. I also built the operator screen and transfer log.

TECHNICAL COMPLEXITY

Users could edit 1C documents after export. Records could also be deleted or duplicated, so the exchange compared prepared data with XML files already sent and let operators resend specific records.

RESULT

The exchange was tested with production-derived data, deployed at one branch, and supported in production while the team prepared further rollouts.

DOCUMENT WORKFLOW

Document management and 1C integration

2022–2023 · Senior 1C Developer · Document management integration

BUSINESS PROBLEM

The document-management system received email and XML data, created sales and return documents in 1C, and exchanged master data. Its SOAP interface also had to move to REST.

MY WORK

I rewrote the DMS API from SOAP to REST, made request headers and parameters case-insensitive, mapped incoming values to 1C reference data, added new XML fields and return rules, and rebuilt the master-data export as a one-action managed form.

TECHNICAL COMPLEXITY

Incoming files could contain missing or mismatched reference values. Database refreshes required the integration to reload messages and restore its initial state. I also added document validation, cache cleanup, consolidated logs, and a log viewer.

RESULT

The integration passed final testing in one branch with document checks and updated loading flows. Operators could trace which 1C documents came from each message and review exchange logs.

PRODUCT CATALOGUE

National product catalogue integration

2025–2026 · Senior 1C Developer · Catalogue integration

BUSINESS PROBLEM

The company could not complete fiscal records or product registration until each item had an identifier from the national catalogue.

MY WORK

I built the 1C register, connected it to the catalogue API, and wrote tools to correct duplicates, import portal errors, and update record status.

TECHNICAL COMPLEXITY

The portal rejected invalid or duplicate identifiers. Staff exchanged error lists in spreadsheets, and fixes had to reach several production 1C databases without overwriting valid records.

RESULT

After rollout, staff could request missing identifiers in bulk and reload corrected rows instead of fixing each product by hand.

Selected independent projects

These projects best represent how I work.

01 / FLAGSHIP

Pomodorough

2026–present · Solo developer · Tagged releases / active development

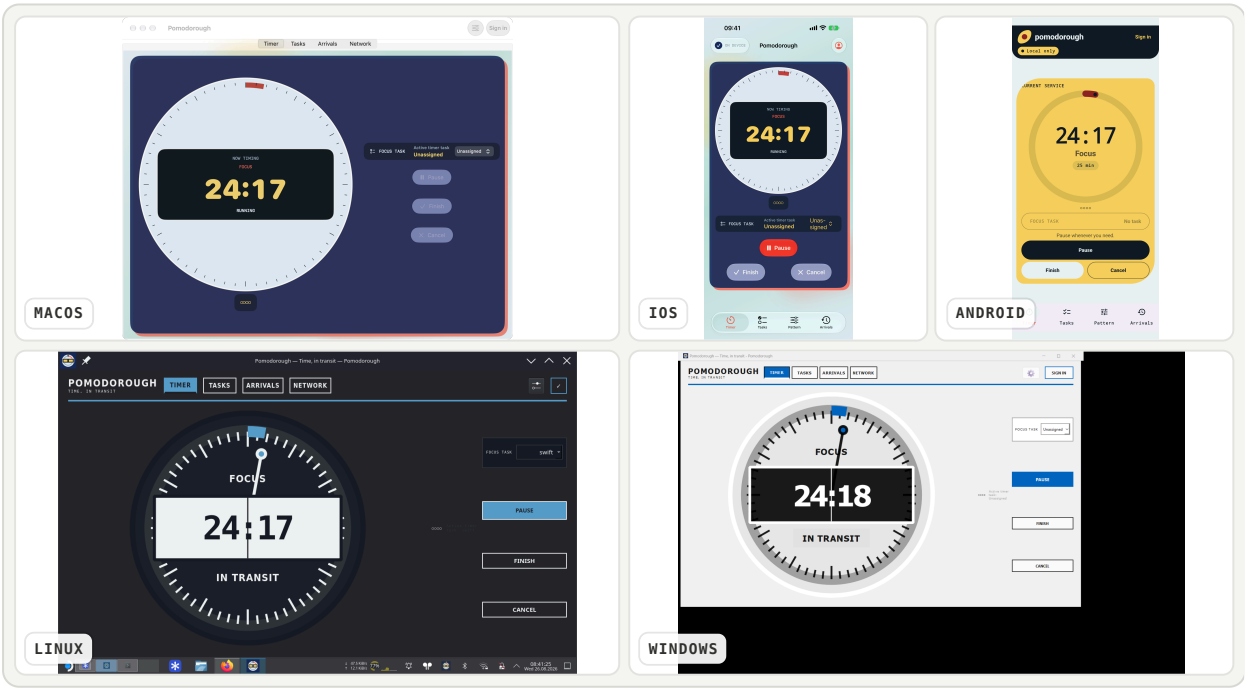
Pomodorough is a timer for seven platforms. It works offline and keeps devices in sync. I built the Go server, the clients, and a Rust/WebAssembly rules engine. The same timer rules run everywhere.

Live	Web/PWA and synchronisation service.
Working development builds	iOS, iPadOS, macOS, Android, Linux, Windows, CLI, and TUI.
Not publicly distributed	No TestFlight, App Store, Play Store, or signed/notarized desktop release yet.

> **Result.** I have working clients on seven platforms, plus CLI and TUI interfaces. The Android suite runs 218 instrumentation tests on API 35 and 36 at two font scales.

`Swift 6` `Kotlin` `Go` `Rust / WASM` `Python / Qt` `SQLite`

Live web app · Source · OpenAPI



02 /
IOS

Struggl

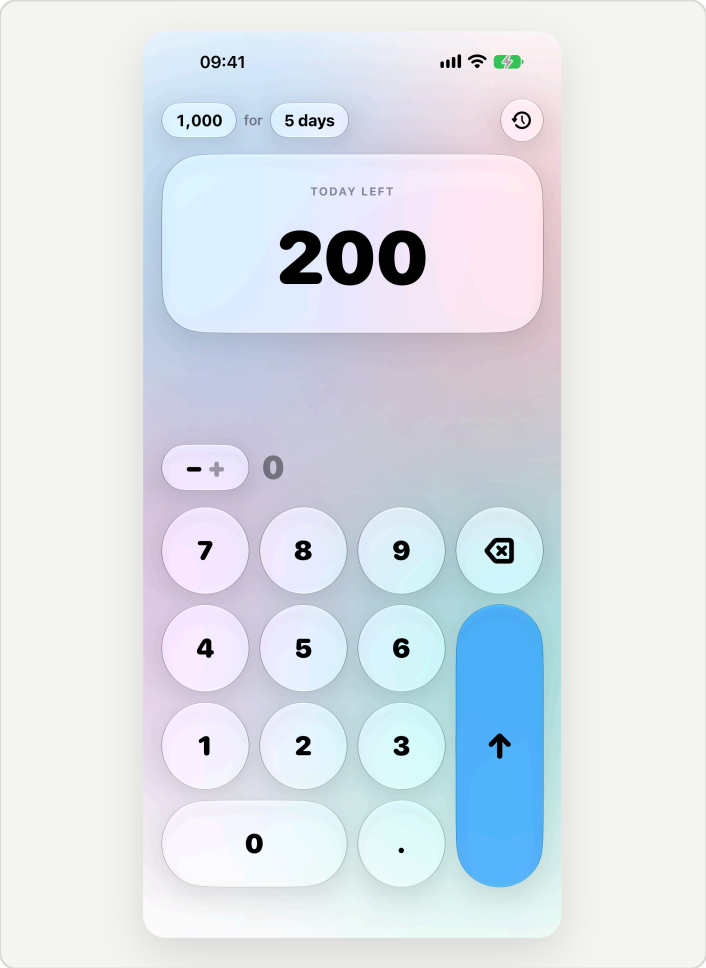
2026 · Solo developer · Working prototype / active development

Struggl tells me how much I can spend today. Before I record a purchase, it shows how that purchase changes the daily budget for the rest of the period. I built the transaction entry, history, deadline changes, and period rollover.

Result. The working iPhone build covers transaction entry, history, deadline changes, and period rollover. Its Swift test targets contain 86 test methods.

SwiftUI · iOS 26 · XcodeGen · GPLv3

Source



03 / IOS

Series

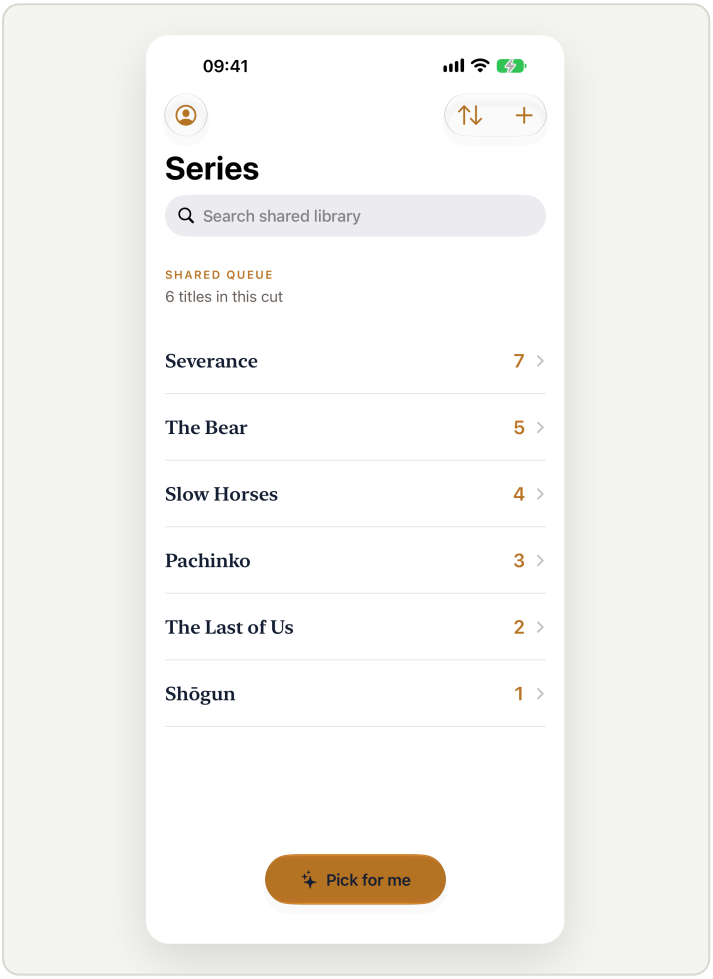
2026 · Solo developer · Working private deployment

Series is an iPhone client for a shared watch-later list. I built the SwiftUI app, local cache, Google sign-in, API client, and test doubles. The saved list still opens when the server cannot be reached.

Result. The app runs as a private, invite-only deployment. Seventeen Swift test methods cover sign-in, pagination, offline reads, and edits.

Swift 6 · SwiftUI · Google Sign-In · REST · Offline reads

Source



04 /
NETWORK

Pinger + MenubarPing

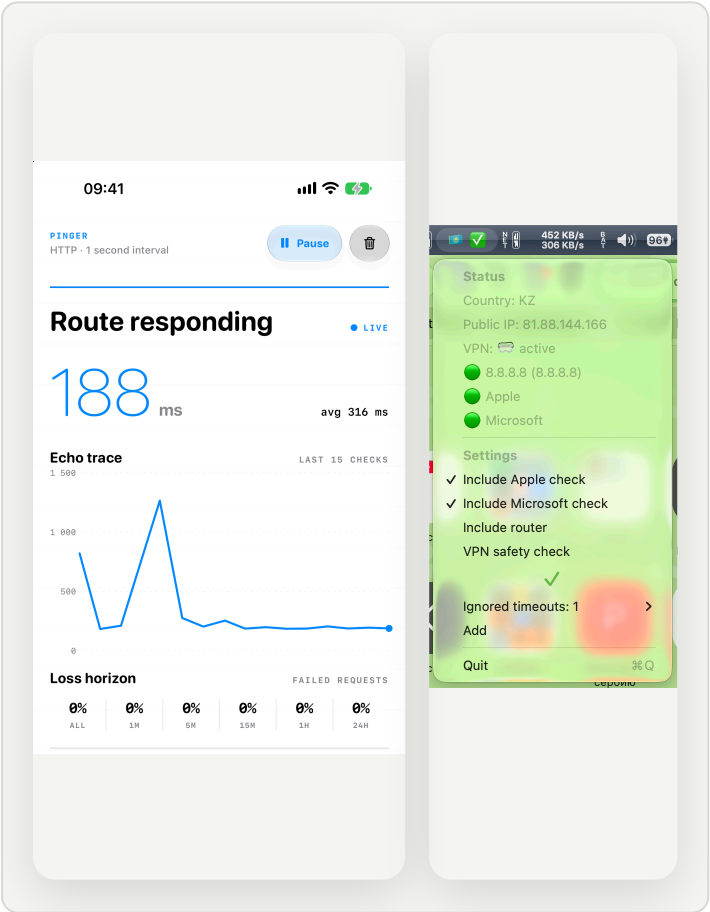
2026 · Solo developer · Pinger prototype / MenubarPing 1.0 tagged

I built both native network monitors. Pinger charts HTTP latency, rolling loss, and route metadata on iPhone. MenubarPing keeps ICMP, HTTP, public IP, country, and VPN safety state visible in the macOS menu bar.

Result. Pinger keeps the latest 300 checks and reports loss over five time windows. MenubarPing has a 1.0.0 tag and an install script.

SwiftUI · Swift Concurrency · iOS · macOS · No runtime dependencies

Pinger source · MenubarPing source



05 / LIBRARY

Commands

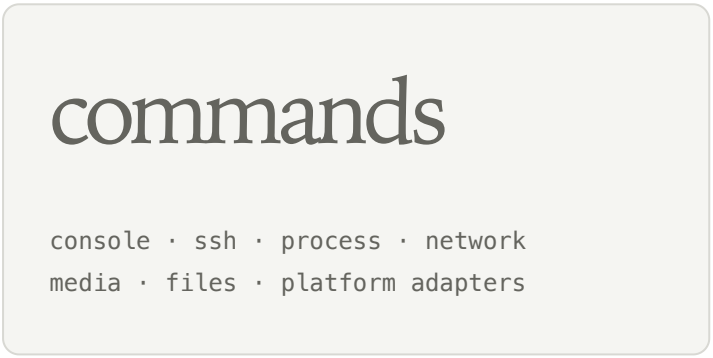
2018–present · Solo developer · Maintained library / active development

Commands began as code copied out of my own automation scripts. I turned it into one Python package for terminal output, SSH, processes, networking, files, downloads, media, and operating-system calls.

> **Result.** The repository has 872 commits since 2018. It has 1,504 test functions across 189 test files, covering 53 source modules.

`Python 3.11+` `Typed package` `Windows / macOS / Linux` `MIT`

Source



Technical capabilities

BUSINESS SYSTEMS	I build and maintain 1C:Enterprise subsystems, operator tools, reports, data exchanges, and integrations with external services.
OFFLINE AND SYNC	Apps save changes before sending them. Failed requests stay queued and retry when the network returns. Newer local work is kept when server state arrives.
NATIVE CLIENTS	I build apps with SwiftUI, Jetpack Compose, and PySide6. I also handle accessibility, OS storage, alarms, notifications, signing, and release packages.
BACKEND	I write Go services backed by SQLite, document REST APIs with OpenAPI, and implement OAuth, token rotation, and server-sent events.
SHARED RULES	The timer rules live in Rust and compile to WebAssembly. Swift, Kotlin, Python, JavaScript, and Go use the same implementation.

GitHub profile · Pomodorough live app · OpenAPI specification